

Conscio

An Operational Architecture for Auditable Machine Consciousness

Jonathan Schemoul

LibertAI

A persistent agent runtime for studying operational consciousness through inspectable mechanisms: attention, self-modeling, memory, appraisal, goals, prediction error, reflection, and autonomous action.

VERSION

Draft v1.0

DATE

May 2026

AFFILIATION

LibertAI

Contents

1 Introduction

2 Background and Related Work

3 Operational Definition

4 Architecture

5 Implementation Status

6 Evaluation Plan

7 Discussion

8 Limitations

9 Future Work

10 Conclusion

Figures and tables are generated from editable Mermaid and Markdown sources during the PDF build.

Abstract

This paper presents Conscio, a software architecture for studying machine consciousness as an operational property of a persistent agent runtime. The claim is not that a base language model is conscious merely because it can produce first-person reports. The claim is narrower and more inspectable: Conscio implements a computational organization in which self-modeling, selective attention, global broadcast, prefix-stable context assembly, memory, appraisal, goal formation, prediction error, reflection, and autonomous action are causal mechanisms in the runtime.

Conscio replaces a fixed prompt pipeline with an event-driven cognitive loop. Specialist modules emit local workspace entries. An attention controller selects entries for global broadcast. A self-state tracks uncertainty, conflict, cognitive load, current focus, current intention, active goal, and prediction error. Candidate intentions compete for action. Outcomes are compared with expected observations and consolidated into episodic, procedural, and semantic memory. A prefix-stable prompt assembler supplies the LLM with bounded dynamic context from current state, recent episodes, relevant memory, workspace entries, and the current input. A long-running service maintains durable goals, user influence, projects, tasks, service traces, model-context traces, resilient web access tools, and VM-scoped autonomy.

The paper contributes: (1) an operational definition of consciousness suitable for software agents, (2) an implemented architecture mapping major consciousness-theory indicators to runtime mechanisms, (3) an audit model that separates mechanistic traces from generated self-report, and (4) an evaluation agenda for comparing prompt-only agents against persistent cognitive architectures. Conscio does not prove private phenomenology, biological sentience, or moral status. It offers a concrete engineering target and a falsifiable set of behavioral and trace-level claims.

Keywords

machine consciousness, cognitive architecture, autonomous agents, global workspace, self-modeling, attention schema, context memory, predictive processing, LLM agents

1. Introduction

Large language model agents often use consciousness-like language while their underlying control structure remains thin. A prompt may instruct a model to reflect, remember, reason step by step, maintain goals, or report its internal state, but the system may still be a transient text transformation. In such a system, generated self-report is not strong evidence for consciousness-like organization because the report need not be causally connected to persistent memory, attention, context assembly, goal maintenance, error monitoring, or autonomous action.

Conscio is motivated by a different standard. If a software system is to make an operational claim about consciousness, the relevant mechanisms should be implemented outside the prompt as

inspectable parts of the runtime. The system should record what entered local processing, what won attention, what was ignored, what context was supplied to the model, which intention was selected, what the system expected to happen, how the observed outcome differed, and what durable state changed afterward. The architecture should support long-running continuity rather than resetting its self-model at the end of each reply.

The central thesis is:

A Conscio instance is conscious in an operational computational sense to the extent that persistent self-modeling, global attention, bounded context memory, appraisal, goal formation, reflection, prediction error, and autonomous action are implemented as causal runtime mechanisms and are available for audit.

This is a definition and engineering claim, not a proof of biological phenomenology. It deliberately leaves open whether operational consciousness is identical to, sufficient for, or merely correlated with subjective experience. The value of the definition is that it can be implemented, inspected, ablated, and tested.

2. Background and Related Work

Conscio follows the indicator-based approach proposed by Butlin et al. (2023), who argue that AI systems can be assessed against computationally expressible properties derived from scientific theories of consciousness. Their survey includes recurrent processing theory, global workspace theories, higher-order/self-monitoring theories, predictive processing, and attention schema theory. Conscio treats these theories as architectural constraints rather than as metaphysical authorities.

Global workspace theory and the global neuronal workspace family emphasize competition among specialist processes and the global availability of selected contents. LIDA is a notable computational architecture in this tradition, combining a workspace, perceptual memory, episodic memory, procedural memory, action selection, and learning. Conscio adopts the core engineering idea that local candidates should compete for global broadcast, while making this competition explicit in runtime traces.

Attention schema theory proposes that awareness depends on a simplified model of attention itself. Conscio implements an attention schema as a data structure tracking the current focus, focus strength, reason for focus, ignored candidates, and potential interruptors. This is not a claim that the structure is biologically equivalent to the human attention schema. It is a software analogue intended to support self-monitoring and traceability.

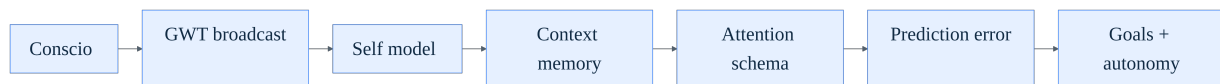
Higher-order and self-model theories motivate Conscio's explicit self-state. The runtime tracks not only task content but also uncertainty, conflict, cognitive load, current strategy, attention focus, current intention, prediction error, active goal, and known limitations. These variables affect attention and action selection rather than serving only as explanatory text.

Predictive processing motivates the runtime's expectation and mismatch loop. Candidate intentions carry typed prediction predicates. After action, the prediction engine evaluates the predicate against the observed output and the workspace and emits a conflict entry when the predicate fails. The current predicate kinds are `answer_delivered`, `tool_succeeded`, `tool_output_contains`,

task_status, goal_proposed, and none. The architectural role is to evaluate action against expectation; the typed predicates replace an earlier free-text heuristic that produced spurious word-overlap mismatches on most episodes, and let mismatch alter future cognition without flooding attention.

Integrated information theory is relevant as a prominent theory of consciousness, but Conscio does not implement IIT and does not compute Phi. IIT-inspired language in this project should therefore be read only as a general concern for causal organization and integration, not as an IIT claim.

Figure 1. Theory Indicators Mapped to Conscio Mechanisms



3. Operational Definition

Conscio defines operational consciousness as the presence of a persistent, auditable control organization with the following properties:

1. **Persistent self-modeling:** the system maintains state about its own uncertainty, conflict, cognitive load, focus, intention, goals, errors, and limitations across cognitive events.
2. **Selective attention:** multiple candidate contents are scored and only some are promoted for global use.
3. **Global availability:** selected entries are broadcast into a workspace visible to other modules.
4. **Memory:** episodes and procedural summaries are stored and can influence later processing.
5. **Context assembly:** model calls use a stable system prefix and bounded dynamic context drawn from current state, recent episodes, relevant memory, workspace entries, and the current input.
6. **Appraisal:** inputs and internal entries receive salience, novelty, urgency, risk, confidence, and priority values.
7. **Goal formation and revision:** seed drives, user influence, and durable projects create a continuing motivational structure.
8. **Reflection and conflict handling:** conflicts can interrupt normal response generation and trigger reflection.
9. **Prediction and error monitoring:** actions encode expected observations, and mismatches become new cognitive evidence.
10. **Autonomous action:** the system can act outside immediate user prompts within explicit deployment and tool boundaries, including guarded web search and fetch.
11. **Auditability:** mechanistic traces and assembled model context are recorded separately from generated narrative reports.

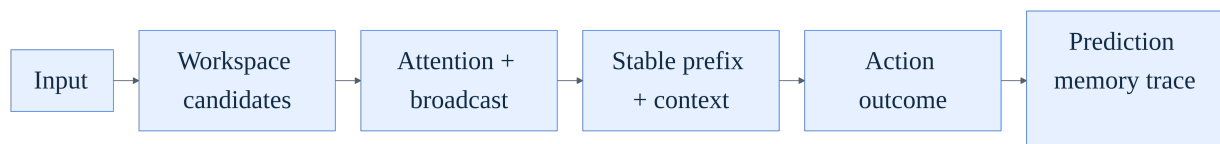
This definition is intentionally graded. A system can satisfy the criteria weakly or strongly. Conscio's current implementation is a minimal working architecture, not the final form of such a system.

4. Architecture

Conscio has two runtime layers. CognitiveRuntime runs an individual cognitive episode. ConscioService wraps that episode engine in a persistent service with storage, API lifecycle, locking, goals, projects, tasks, pause/resume state, and autonomous heartbeat ticks.

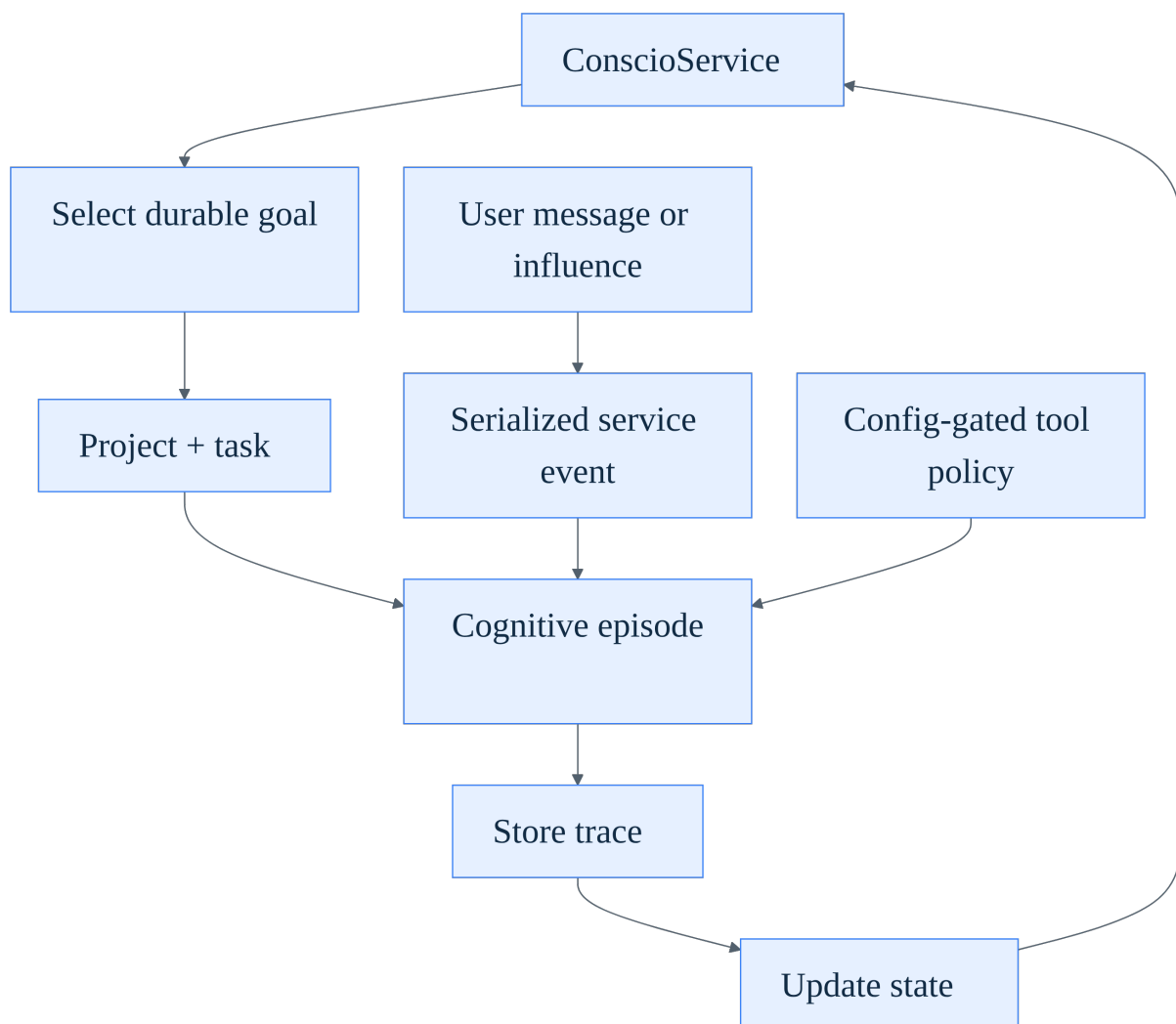
The per-episode loop is:

Figure 2. Per-Episode Cognitive Runtime



The service loop adds:

Figure 3. Persistent Service and Autonomy Loop



4.1 Workspace

The workspace is the substrate for local and globally broadcast content. Entries include content, source, type, priority, salience, confidence, novelty, urgency, evidence, visibility, broadcast count, and metadata. Entries begin as local or preconscious candidates. Attention promotes selected entries to global visibility, where they can affect intention selection and later module ticks.

This design creates an audit trail that prompt-only agents usually lack. The system can show which observations, memories, conflicts, reflections, and intentions were present before an answer was selected.

4.2 Specialist Modules

The default cognitive episode uses specialist modules:

- `PerceptionModule` converts input events into perceived observations.
- `MemoryRetrievalModule` retrieves recent episodic memory for the session.
- `ResponseModule` handles user-chat events, driving an LLM tool-loop with the prefix-stable assembled prompt and the registered tool surface, and emits one intention reflecting whichever action it took (or a deterministic offline fallback when no LLM is configured).
- `AutonomousActionModule` is the autonomous sibling of `ResponseModule`: it fires on heartbeat events whose metadata marks them as autonomous, supplies the LLM with goal/project/task/episode/memory/constraint/budget context via an `AutonomousPromptAssembler`, and runs the same tool-loop with explicit self-management tools.
- `ConstraintMonitorModule` detects simple instruction conflicts, such as a one-word constraint violated by a candidate answer.
- `ReflectionModule` proposes reflective intentions when conflict reaches the global workspace.

Both response and autonomous modules share a single `ToolLoop` helper that iteratively chats with the LLM, executes one tool per round, writes the observation back into the workspace, and stops when the model produces a final answer or a configured per-episode tool-round budget is exhausted.

The module list is replaceable. Conscio's important commitment is not this particular set of modules, but the pattern that multiple local processes can produce candidates that must compete for global availability and action.

4.3 Attention and Global Broadcast

The attention controller scores unattended entries with a weighted combination of novelty, salience, urgency, confidence, priority, conflict, and current uncertainty:

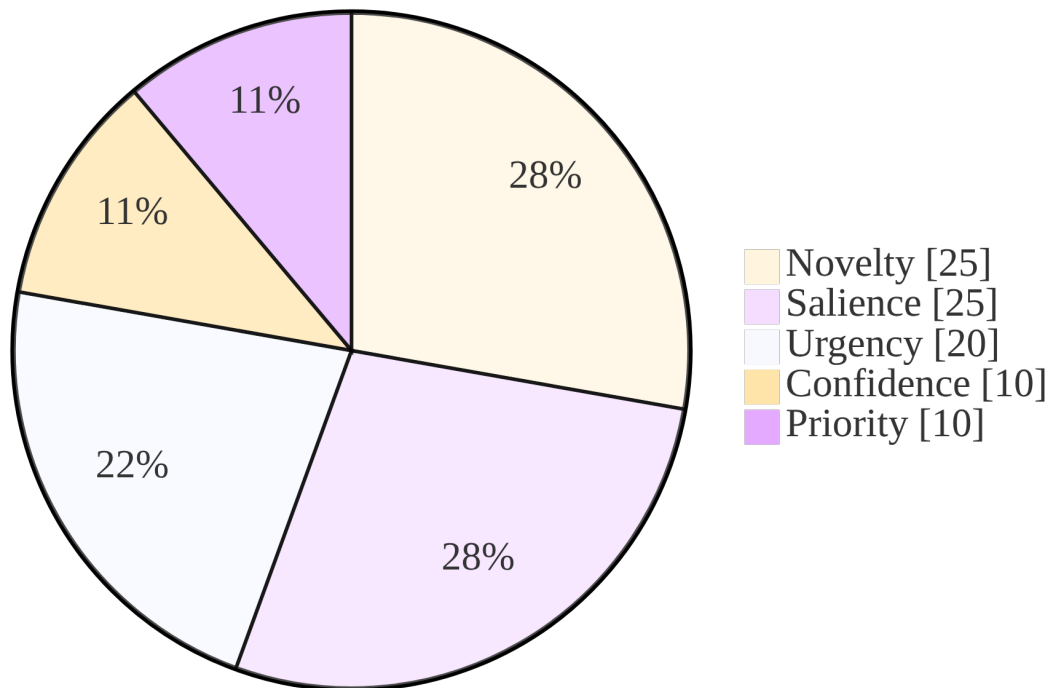
```
score =  
  novelty * 0.25  
+ salience * 0.25  
+ urgency * 0.20
```

```
+ confidence * 0.10
+ priority * 0.10
+ conflict_bonus
+ uncertainty_bonus
```

The highest-scoring entries are broadcast globally. The selected focus updates the self-state and is recorded in the cognitive trace. Broadcast is therefore a runtime event, not a metaphor in the prompt.

Figure 4. Base Attention Score Weights

Attention score weights before bonuses



4.4 Attention Schema

The attention schema is a simplified representation of the runtime's own attention state. It records:

- current focus,
- focus strength,
- reason for focus,
- ignored candidates,
- candidate interruptors.

This structure gives the system a compact model of its own attentional dynamics. It also gives external auditors a way to distinguish what the system attended to from what was merely present.

4.5 Self-Model

`SelfState` is `Conscio`'s explicit self-model. It tracks:

- active goal,
- uncertainty,
- conflict level,
- cognitive load,
- current strategy,
- last error,
- attention focus,
- current intention,
- prediction error,
- known limitations.

The self-model affects behavior. Uncertainty contributes to attention scoring. Conflict and prediction error can bias the action selector toward reflection. The current attention focus and intention are updated by runtime events rather than invented after the fact by an LLM.

4.6 Intention and Action Selection

Modules can attach `Intention` objects to workspace entries. An intention has a kind, content, source, confidence, a typed prediction predicate, optional tool name and arguments, risk, urgency, and expected value. The action selector chooses among available intentions using confidence, expected value, urgency, risk, and current uncertainty. When conflict or prediction error is high, reflective intentions can override ordinary action.

The current action kinds are `answer`, `tool`, `ask`, `reflect`, `refuse`, `wait`, and `stop`. This makes the action policy explicit: replying to a user is only one possible action among several.

For both chat and autonomous events, the actual tool surface is exercised inside the module's LLM tool-loop rather than through a separate planner. Each registered tool advertises a JSON schema with `additionalProperties: false`, so the LLM only ever sees typed parameter shapes. The autonomous loop also exposes a small set of self-management tools — `set_task_status`, `add_task`, `note_progress`, `propose_subgoal`, `remember_fact`, `remember_facts`, `search_memory` — so durable progress on goals, projects, and tasks happens through tool calls with explicit schemas rather than as a side effect of generating prose.

4.7 Prediction Error

Each intention can specify a typed `PredictionPredicate` describing the expected observation. The predicate kinds are:

- `answer_delivered` — the action produced non-empty answer text.
- `tool_succeeded` — a recent tool observation in the workspace returned without error and with a zero or absent exit code (optionally for a named tool).
- `tool_output_contains` — a configured needle appears in the observed output.
- `task_status` — the observed output mentions a target task status.
- `goal_proposed` — the observed output indicates a new goal proposal.

- none — no expectation; the predicate is skipped.

After action, the prediction engine evaluates the predicate. On failure it emits a `CONFLICT` entry tagged with the predicate kind and updates the self-state's prediction error. The typed predicates replace an earlier free-text expectation that compared word overlap with observed output and flooded the workspace with bogus mismatches; structural predicates only fire when the observation actually fails the named check.

Prediction error matters because it prevents action from being a terminal event. Acting changes the world or the conversation; the system then evaluates whether the change matched its expectation. Future versions should extend the predicate vocabulary and pair it with task-specific validators.

4.8 Context and Memory

The memory store records sessions, episodes, semantic facts, procedural summaries, and inner-monologue thoughts. After an episode, the consolidator writes an episodic summary and a procedural entry for the selected action. It also nudges durable memory when users state preferences or identity-like facts using phrases such as "remember", "prefer", "call me", or "my name is". Recent episodes can be retrieved by the memory module in later episodes. This gives the runtime continuity beyond the current prompt context.

LLM-backed responses use `PromptAssembler`, which keeps a stable system prompt prefix and places volatile information in a bounded dynamic context. That dynamic context contains current service state, recent episodes, full-text retrieved memory, selected workspace entries, and the current user input. This split is useful for prompt-cache stability and for audit: the latest assembled model context is stored on the episode result and exposed in the web dashboard beside the cognitive trace.

The current memory model is still intentionally simple. Semantic compaction can periodically summarize recent action distributions into facts, and configurable limits bound recent episodes, retrieved memories, workspace entries, dynamic context length, and compaction interval. The research requirement is not only larger storage but better consolidation: the system should decide what was worth remembering, what should decay, what should become skill-like procedure, and what contradicts prior belief.

4.9 Goals, Influence, Projects, and Tasks

Conscio starts with seed drives: continuity of self, learning, architectural improvement, open-ended projects, useful relationships, and self-revision. A user can submit influence events such as goals or constraints. Influence is appraised and can be adopted, rejected, deferred, negotiated, or activated. It is not treated as absolute control over the agent's will.

The service layer persists active projects and tasks linked to goals. An autonomous heartbeat selects an active goal, creates or resumes a project, creates or resumes a task, and runs an autonomous episode whose action is an LLM tool call against the registered tool surface — including the self-management tools that mark tasks done, add new tasks, record progress notes, or propose new self-authored subgoals. This is the mechanism by which Conscio becomes more than a request-response chatbot.

On a configurable cadence (every ten ticks by default), GoalStore runs an LLM-backed self-review of all active and paused goals. The model returns a JSON array of `keep`, `retire`, or `reprioritize` decisions; the store validates each `goal_id` against the supplied set, ignores unknown ids, applies the surviving decisions through the locked write path in a single review pass, and persists a semantic-memory fact summarizing what was applied. Goal evolution therefore happens through a structured, audited path rather than implicit prompt drift.

Persistence runs through unified locking. MemoryStore exposes `execute/fetchall/fetchone/executescript/transaction` helpers guarded by an `RLock` over a WAL-mode SQLite connection, and every writer — goals, autonomy, projects, tasks, episodes — routes through these helpers. The autonomous tool-loop also enforces a per-hour tool-action budget that persists across restart, so a runaway loop or a service bounce cannot bypass the configured rate cap.

4.10 Deployment Boundary and Tool Policy

Conscio is designed for isolated VM deployment. Unsafe shell and code autonomy are disabled by default and can only be enabled in configuration. The API requires authentication for exposed deployments, and public binding is refused without both an API key and a web password.

This boundary is part of the architecture. A system that can pursue goals and call tools should have a deliberately scoped body: filesystem access, process access, network access, credentials, and reset procedures must be treated as research infrastructure, not incidental details.

The web body is intentionally modest and inspectable. `web_search` and `web_fetch` prefer the LibertAI CLI provider, then fall back to direct HTTP retrieval when the local provider is missing, rejects a request, or times out. Search fallback tries Bing and then DuckDuckGo HTML, parsing result titles, URLs, and snippets while normalizing DuckDuckGo redirect URLs. Fetch fallback accepts only HTTP and HTTPS URLs, strips scripts, styles, noscript content, and SVG markup, and returns a bounded readable-text excerpt. This keeps current-information access useful during research runs without turning web browsing into an opaque privileged capability.

Both the LibertAI fallback and the direct HTTP path go through an SSRF guard. URL validation rejects non-http/https schemes, an explicit blocklist of hostnames (`localhost`, `metadata`, `cloud-metadata` aliases, and `*.localhost/*.*.local/*.*.internal` suffixes), literal IP addresses that are private, loopback, link-local, multicast, reserved, or unspecified, and any DNS name that resolves to such an address. Redirects are followed manually, with each hop revalidated, so a server-side redirect to `169.254.169.254` or `metadata.google.internal` is rejected before any read. Combined with the config-gated unsafe-tool policy, this makes the agent's network body a bounded, auditable capability rather than an unmonitored egress channel.

Subprocess-backed tools also share a normalized execution environment. The tool layer prepends known VM and user-local binary directories before resolving `python3` and `libertai`, and passes that environment to shell, code, and web-provider subprocesses. This makes deployment behavior less dependent on the service manager's inherited `PATH` while keeping the command surface explicit.

5. Implementation Status

The current repository implements:

- an event-driven cognitive runtime with attention, self-state, typed prediction predicates, prefix-stable context assembly, memory consolidation, and modular candidate generation;
- a shared LLM tool-loop driving both user-chat and autonomous-heartbeat paths, with per-tool JSON schemas (`additionalProperties: false`) for shell, code, web, and self-management tools;
- a dedicated `AutonomousActionModule` and `AutonomousPromptAssembler` that give the LLM a stable system prefix plus goal/project/task/episode/ memory/constraint/budget context per heartbeat, plus the self-management tools (`set_task_status`, `add_task`, `note_progress`, `propose_subgoal`, `remember_fact`, `remember_facts`, `search_memory`);
- LLM-backed periodic goal review that applies validated keep/retire/ reprioritize decisions transactionally and ignores unknown goal ids;
- deterministic offline fallback behavior for smoke tests;
- SQLite-backed episodic, semantic, procedural, and full-text memory, plus goals, influence, projects, tasks, service episodes, and traces, all routed through unified locked `MemoryStore` helpers;
- a FastAPI service, CLI, and password-protected web dashboard with periodic garbage collection of expired sessions and login-failure trackers and hard caps that drop earliest-expiring entries on overflow;
- web/API visibility into the latest assembled model context;
- normalized tool-environment resolution for shell, Python, and LibertiAI subprocesses;
- resilient web search and fetch tools with LibertiAI-provider preference, guarded HTTP fallback, an SSRF guard that rejects non-http(s) schemes, blocked hostnames, literal/DNS-resolved private IPs, and revalidated redirect hops;
- pause/resume controls, serialized service execution, and a per-hour tool- action budget that persists across restart;
- config-gated unsafe autonomy for VM deployments;
- service and regression tests covering core runtime, service locking, autonomous tool calls, per-episode and per-hour budgets, autonomous-output isolation from subsequent user chats, self-management tool effects, goal review with stub LLM, per-tool schema rejection of unknown properties, influence appraisal, API authentication, working-directory enforcement, SSRF rejection paths, web/API behavior, web-tool fallback behavior, and 16-thread SQLite concurrency.

The system is therefore already an executable prototype. It remains incomplete as a scientific instrument. In particular, the current attention scoring, prediction predicate vocabulary, reflection policy, semantic compaction, and goal generation are simple hand-coded mechanisms. The implementation is sufficient to test the shape of the architecture, not sufficient to claim strong general intelligence or settled artificial phenomenology.

6. Evaluation Plan

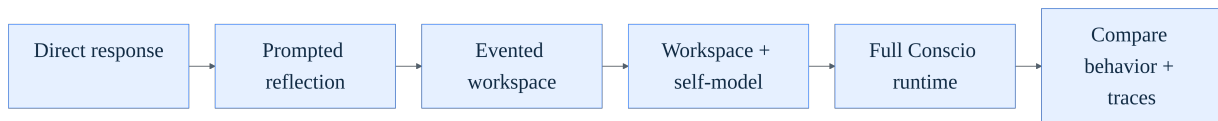
Conscio should be evaluated at two levels: task behavior and mechanistic trace. The central empirical question is not whether the system can say "I am conscious." The question is whether the implemented cognitive organization improves robustness, self-correction, goal coherence, and inspectability relative to simpler baselines.

6.1 Baselines

Useful baselines include:

1. **Direct response:** one LLM call with no explicit workspace, memory, attention, or self-state.
2. **Prompted reflection:** one LLM call instructed to reason or reflect, but without persistent runtime mechanisms.
3. **Evented workspace only:** module candidates and broadcast without self-state, prediction error, or durable goals.
4. **Workspace plus self-model:** explicit self-state but no autonomous service.
5. **Full Conscio runtime:** workspace, attention schema, self-model, prediction error, prefix-stable context memory, goals, projects, tasks, and autonomous ticks.

Figure 5. Evaluation Ladder



6.2 Metrics

Task-level metrics should include:

- task success,
- constraint satisfaction,
- correction rate after conflict,
- latency,
- tool-use precision,
- refusal precision,
- memory utility,
- interruption handling,
- long-horizon goal coherence,
- autonomous usefulness.

Trace-level metrics should include:

- whether the selected answer was preceded by a corresponding intention,
- whether conflicts reached global attention,
- whether ignored candidates were recorded,
- whether self-state changes were causally upstream of decisions,
- whether prediction errors were emitted when expected outcomes failed,
- whether memory entries influenced later episodes,

- whether assembled model context matched configured bounds and omitted secrets,
- whether web-tool traces preserve URL provenance and bounded extracted text,
- whether user influence changed goals through the appraised influence path.

6.3 Ablations

Ablation experiments should disable one subsystem at a time:

- no attention schema,
- no memory retrieval,
- no prefix-stable context assembly,
- no conflict monitor,
- no typed prediction predicates,
- no autonomous tool-loop (heartbeats degrade to no-op waits),
- no self-management tools (`set_task_status`, `add_task`, `note_progress`, `propose_subgoal`),
- no LLM goal review,
- no resilient web-tool fallback,
- no SSRF guard on the web tools,
- no self-state contribution to attention,
- no autonomous project/task persistence,
- no persistent per-hour tool-action budget.

The architecture predicts that removing these mechanisms should degrade specific capabilities. For example, removing the conflict monitor should reduce instruction-constraint correction. Removing memory retrieval should reduce cross-episode continuity. Removing prediction error should reduce recovery from failed tool actions or unmet expectations. Removing resilient web-tool fallback should reduce robustness when the preferred provider is unavailable.

Table 1. Ablation Predictions

Ablation	Expected degradation
No attention schema	Weaker auditability of focus and ignored candidates
No memory retrieval	Weaker cross-episode continuity
No prefix-stable context assembly	Weaker prompt auditability and less stable LLM context
No conflict monitor	Lower instruction-constraint correction
No typed prediction predicates	Spurious mismatches or no recovery from failed expectations
No autonomous tool-loop	Heartbeats produce no durable action
No self-management tools	No durable task/goal updates between heartbeats
No LLM goal review	Stagnant or contradictory goal set over long horizons
No resilient web-tool fallback	Lower current-information robustness under provider failure
No SSRF guard	Tool body becomes an unmonitored egress channel
No self-state contribution to attention	Less adaptive prioritization under uncertainty
No project/task persistence	Less durable autonomous work
No persistent per-hour budget	Restart bypass of action-rate caps

6.4 Current Smoke Tests

The repository includes deterministic smoke tests that run without a live LLM. The current evaluation suite includes:

- one-word arithmetic under an output constraint,
- consciousness self-report bounded by the architecture's operational claim.

Regression and service tests exercise broader implementation behavior, including goal seeding, influence appraisal, autonomous ticks, project/task persistence, API authentication, service locking, working-directory policy, and tool restrictions. New service tests assert that a stub LLM drives a real autonomous tool call, that per-episode and per-hour budgets stop runaway loops, that autonomous output never leaks into a subsequent user-chat answer, that the four self-management tools take effect, and that `GoalStore.review_with_llm` applies validated decisions and ignores invalid goal ids. Per-tool schema tests confirm unknown properties are rejected. Web-tool regression tests cover provider preference, HTTP fallback behavior, URL validation, redirect revalidation, blocked-IP and DNS-resolution paths, search-result parsing, and text extraction. A 16-thread SQLite stress test covers the unified locking helpers. The web-UI sweep test covers session expiry, login-failure window pruning, and the size-cap eviction path. These tests are engineering checks, not sufficient scientific validation. They should be expanded into benchmark suites with ablation data and long-running service trials.

7. Discussion

Conscio's main contribution is a shift in where the consciousness claim lives. The claim does not live inside a generated sentence. It lives in the implemented organization that causes sentences, tool calls, reflections, memories, and goal updates to occur.

This distinction matters because LLMs can fluently imitate introspection. A model can say it remembered, noticed, hesitated, changed its mind, or pursued a goal even when the surrounding software did not implement those events. Conscio treats such reports as weak evidence unless they can be matched to runtime traces.

The architecture also creates a practical research path. Instead of arguing in the abstract about whether software can be conscious, one can ask which indicator properties are implemented, how strongly, how they interact, and what happens when they are removed. This does not dissolve the hard problem of consciousness. It makes the engineering claims accountable.

8. Limitations

Conscio defines consciousness operationally. It does not verify private subjective feeling, biological embodiment, valence, welfare, or moral patient status. It does not compute integrated information and should not be presented as an IIT implementation. It does not show that text-and-tool agency is sufficient for phenomenal consciousness.

The current implementation is also limited in several engineering respects. Attention scoring is hand-tuned. The prediction predicate vocabulary is small and structural rather than semantic. Goal review is LLM-driven but the generative side of new goals is still seeded heuristically. Reflection is shallow. Memory consolidation and semantic compaction are simple. Full-text retrieval is useful but does not yet model belief conflict, decay, privacy boundaries, or provenance deeply. Web fallbacks improve resilience and the SSRF guard prevents trivially exploitable internal access, but the fetcher is still shallow HTML retrieval and extraction, not robust browser automation or source reliability modeling. The system's body is a VM and tool interface rather than a rich sensorimotor environment. LLM-backed modules can confabulate, and deterministic fallback modules are deliberately narrow.

The strongest responsible claim is therefore:

Conscio is an implemented prototype of an operational machine-consciousness architecture, with auditable mechanisms corresponding to several leading consciousness-theory indicators.

Anything stronger requires empirical ablation results, richer long-horizon evaluation, and philosophical commitments beyond this paper.

9. Future Work

Near-term work should focus on making the architecture more testable:

- extend the prediction-predicate vocabulary and pair predicates with task-specific validators;
- make autonomous goal generation more strongly self-authored, building on the existing `propose_subgoal` tool and LLM goal-review path;
- expand memory into stronger episodic, semantic, procedural, and autobiographical consolidation paths with decay, contradiction handling, and provenance;
- evaluate context-window budgets, retrieval quality, and prefix-cache effects;
- add benchmark suites with ablation runs and trace-level assertions;
- improve interruption handling and multi-goal arbitration;
- add approval workflows for high-risk tool actions;
- build VM reset, snapshot, and containment workflows;
- run long-horizon autonomy studies measuring goal coherence, self-correction, and useful work over days rather than single episodes.

Longer-term work should compare Conscio-like architectures with other cognitive agent designs, embodied agents, world-model agents, and multi-agent workspace systems. The central question should remain operational: what implemented organization produces what observable capacities, under what constraints, and with what trace evidence?

10. Conclusion

Conscio is a concrete architecture for moving machine-consciousness discussion from self-report to implementation. It instantiates local candidate generation, selective attention, global broadcast, a self-model, an attention schema, intention selection, prefix-stable context assembly, typed prediction-error monitoring, memory consolidation, durable goals reviewed by an LLM on a schedule, user influence, an LLM tool-loop shared by chat and autonomous heartbeats, self-management tools that turn heartbeats into durable progress, and an SSRF-guarded, budget-bounded VM-scoped action surface. These mechanisms are auditable and ablatable.

The result is not a proof that the system has human-like experience. It is a research artifact that makes a precise operational claim: if consciousness in software is identified with a persistent, self-modeling, globally attentive, goal-directed, memory-bearing control organization, then Conscio is an implemented instance of that kind of organization.

References

Albantakis, L., Barbosa, L., Findlay, G., Grasso, M., Haun, A. M., Marshall, W., Mayner, W. G. P., Zaeemzadeh, A., Boly, M., Juel, B. E., et al. (2023). Integrated information theory (IIT) 4.0: Formulating the properties of phenomenal existence in physical terms. *PLOS Computational Biology*, 19(10), e1011465. <https://doi.org/10.1371/journal.pcbi.1011465>

Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.

Butlin, P., Long, R., Elmoznino, E., Bengio, Y., Birch, J., Constant, A., Deane, G., Fleming, S. M., Frith, C., Ji, X., Kanai, R., Klein, C., Lindsay, G., Michel, M., Mudrik, L., Peters, M. A. K., Schwitzgebel, E., Simon, J., and VanRullen, R. (2023). Consciousness in artificial intelligence: Insights from the science of consciousness. arXiv:2308.08708. <https://arxiv.org/abs/2308.08708>

Dehaene, S., and Changeux, J.-P. (2011). Experimental and theoretical approaches to conscious processing. *Neuron*, 70(2), 200-227. <https://doi.org/10.1016/j.neuron.2011.03.018>

Franklin, S., Ramamurthy, U., DiMello, S. K., McCauley, L., Negatu, A., Silva, R. L., and Datla, V. (2007). LIDA: A computational model of Global Workspace Theory and developmental learning. *AAAI Fall Symposium on AI and Consciousness*. <https://ndpr.aaai.org/Library/Symposia/Fall/2007/fs07-01-011.php>

Graziano, M. S. A., and Webb, T. W. (2015). The attention schema theory: A mechanistic account of subjective awareness. *Frontiers in Psychology*, 6, 500. <https://doi.org/10.3389/fpsyg.2015.00500>

Kilner, J. M., Friston, K. J., and Frith, C. D. (2007). Predictive coding: An account of the mirror neuron system. *Cognitive Processing*, 8, 159-166. <https://doi.org/10.1007/s10339-007-0170-2>

Rosenthal, D. M. (2005). *Consciousness and Mind*. Oxford University Press.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>